

Reproducing Self-Distillation Fine-Tuning on Tool Use: A Running Report with a Released Model

Ahmed Khaled Khamis* Xiaotong Ji Rasul Tutunov Haitham Bou-Ammar

May 29, 2026

Abstract

Self-Distillation Fine-Tuning (SDFT) was recently proposed as an on-policy distillation method in which a student model is trained on its own generations while matching a demonstration-conditioned teacher derived from the same model. Despite released code, several users reported difficulty reproducing the reported Tool Use results. In this note, we document a successful reproduction of SDFT on the released Tool Use split using Qwen2.5-7B-Instruct. We find that reproducing the result requires matching several implementation details that are easy to miss, including the use of per-token forward KL rather than the reverse KL suggested by the paper equation, the exact teacher-prompt construction, EMA teacher updates, skipped completion tokens, and the evaluation length. With these settings, our checkpoint reaches 70.1% accuracy on the held-out Tool Use split, compared with 70.6% reported by Shenfeld et al. and 42.2% for the base model. We release the fine-tuned model, evaluation outputs, training logs, and the full configuration to provide a working reference point for future studies of SDFT.

1 Introduction

Self-Distillation Fine-Tuning (SDFT) is an appealing idea: instead of training only on static expert demonstrations, a model generates its own completions and is then updated to match a teacher distribution conditioned on a demonstration (Shenfeld et al., 2026). This places SDFT in the broader family of on-policy distillation methods, where the training distribution follows the student’s own behaviour (Kim and Rush, 2016; Agarwal et al., 2024b,a; Zimmer et al., 2025; ?).

However, reproducing SDFT requires care. The public implementation contained several choices that are not immediately obvious from the paper description alone. In particular, the reported experiments use a per-token forward KL objective, an EMA-updated teacher, a specific demonstration-conditioned teacher prompt, and an evaluation protocol with long greedy decoding. Several community reports have noted difficulty reproducing the published numbers when these details are not matched (devonbrackbill, 2026; yufeng-04, 2026; Proteusiq, 2026).

This note provides a minimal successful reproduction of SDFT on the released Tool Use split. Using Qwen2.5-7B-Instruct and the authors’ trainer, we obtain 70.1% accuracy on the held-out Tool Use set, closely matching the reported 70.6% and substantially improving over the 42.2% base model. Our goal is not to introduce a new algorithm, but to make the SDFT recipe executable and inspectable. We therefore release the fine-tuned checkpoint, evaluation outputs, training logs, and the exact configuration used for the run.

*Affiliations and contact: Ahmed Khaled Khamis, akhamis6@gatech.edu; Xiaotong Ji, Imperial College London; Rasul Tutunov, rtutunov@outlook.com; Haitham Bou-Ammar, UCL Centre for AI.

Table 1: Replication details

Detail	What matters	Why it matters
KL direction	Forward KL, $\alpha=0$	Matches reported runs (hank0316 and idanshen, 2026)
Teacher input	Prompt + golden response + answer-on-your-own instruction	Defines the distillation target
Student input	Prompt only	Keeps rollout on-policy
Teacher update	EMA Teacher, $\alpha_{\text{ref}}=0.01$	Teacher is not fixed
Rewards	Always Zero	Inherited GRPO logging, not used (bhavitvyamalik, 2026)
Eval	Greedy, 2048 tokens	Train uses 1024; longer eval gives room for tool traces
Skip tokens	First 3 completion tokens	Matches trainer recipe; masking phrases like "Based on the text..." or "Following the example..."

2 Background

On-policy distillation. Classic distillation usually means a fixed teacher and a student trained on *someone else’s* inputs—often static prompts or a pre-recorded corpus. **On-policy distillation** instead matches the teacher on tokens the **student** actually samples, $y \sim \pi_\theta(\cdot \mid x)$, so the training distribution tracks the policy you care about at deployment (Agarwal et al., 2024b). That cuts the off-policy gap you get from plain SFT on demonstrations alone: the loss is evaluated on the student’s own rollouts, not only on frozen expert strings. Recent work formalizes this as learning from self-generated trajectories with a guiding distribution (Agarwal et al., 2024b), where SDFT sits in the same family, but builds the teacher from the *same* base weights plus an in-context demonstration c rather than a separate reward model or a permanently frozen checkpoint.

2.1 Self-distillation fine-tuning

For each training example, we observe a task prompt x together with an expert demonstration c , such as a golden tool-call trace. The method defines two conditional policies. The *student* policy, π_θ , is conditioned only on the task prompt x . The *teacher* policy, π_ϕ , is conditioned on both the task prompt and the expert demonstration, (x, c) , using a fixed prompting template. Intuitively, the teacher is allowed to inspect the demonstration before producing its own answer, whereas the student must solve the task from x alone.

Training is performed on-policy with respect to the student. First, a completion

$$y = (y_1, \dots, y_T) \sim \pi_\theta(\cdot \mid x) \quad (1)$$

is sampled from the student. Then, at each completion position t , the method compares the student next-token distribution

$$\pi_\theta(\cdot \mid x, y_{<t}) \quad (2)$$

with the teacher next-token distribution

$$\pi_\phi(\cdot \mid x, c, y_{<t}). \quad (3)$$

Thus, the two models are evaluated on the same student-generated prefix $y_{<t}$, but the teacher additionally has access to the expert demonstration c . The teacher parameters ϕ are not kept fixed: they are updated as an exponential moving average (EMA) of the student parameters θ .

At the sequence level, Shenfeld et al. (2026) writes the objective as

$$\mathcal{L}(\theta) = D_{\text{KL}}(\pi_{\theta}(\cdot | x) \| \pi_{\phi}(\cdot | x, c)). \quad (4)$$

In practice, this sequence-level expression is implemented as a sum of token-level divergences over the generated completion:

$$\mathcal{L}_{\text{tok}}(\theta) = \sum_{t=1}^T m_t D_{\text{KL}}(\pi_{\theta}(\cdot | x, y_{<t}) \| \pi_{\phi}(\cdot | x, c, y_{<t})), \quad (5)$$

where $m_t \in \{0, 1\}$ denotes an optional token mask. The implementation applies this divergence over the full vocabulary at each completion position, while also supporting masking of the first few completion tokens and truncated importance sampling.

3 Replication Details

The main challenge in reproducing SDFT was not the high-level idea, but matching the executable recipe used by the released implementation. Several details that affect the Tool Use result are easy to overlook: the construction of the demonstration-conditioned teacher prompt, the effective KL direction in the trainer, the EMA update of the teacher model, masking choices, and inherited GRPO logging fields that are not used by the distillation objective. We collect these details here as a practical reference for others attempting to reproduce or extend SDFT.

In the released Tool Use setup, each example provides a task prompt and a golden response. The rollout prompt contains only the task prompt, while the teacher prompt appends the golden response as an example trace before asking the model to produce its own response. This separation between the student rollout context and the teacher conditioning context is one of the main details to preserve when reproducing the result.

3.1 Demonstrations and in-context learning

SDFT differs from standard supervised fine-tuning in how it uses the demonstration (c). The student is rolled out on the task prompt (x) alone, producing completions from $(\pi_{\theta}(\cdot | x))$. The demonstration is used only on the teacher side: the teacher receives a constructed `teacher-prompt` containing the task text, the golden response, and an instruction to answer the original task independently. The resulting distribution $(\pi_{\phi}(\cdot | x, c))$ is then used as the distillation target on the student’s own sampled prefixes. Consequently, the teacher prompt is not an incidental formatting choice. If the demonstration is malformed, weak, or inserted using a template that the base model does not follow reliably, the teacher distribution may provide a poor training signal even though the rest of the SDFT pipeline is implemented correctly. In the released Tool Use setup, each example provides a task prompt and a golden response. The rollout prompt contains only the task prompt, while the teacher prompt appends the golden response as an example trace before asking the model to produce its own response. In our reproduction, this teacher prompt followed the template:

```
$orig_content — This is an example for a response to the question:
$output_text — Now answer with a response of your own, including the
thinking process.
```

This separation between the student rollout context and the teacher conditioning context is one of the main details to preserve when reproducing the result. Rollouts use `prompt` only, whereas the teacher forward pass uses `teacher-prompt`. This makes the quality of the demonstration field a reproducibility-critical detail. In Tool Use, the golden response is not merely metadata: it defines the behaviour that the teacher distribution exposes to the student. Malformed tool names, inconsistent formatting, or broken `Action: / Action Input:` structure in the golden response can therefore degrade the teacher target even when the training loop itself is unchanged.

A related assumption is that the base model can already benefit from the demonstration in context. Since the teacher is obtained from the same model weights with (c) added to the prefix, the demonstration is useful only to the extent that it changes the model’s next-token distribution in the desired direction. If the base model ignores the demonstration, then the teacher and student distributions may be nearly identical, $(\pi_\phi(\cdot | x, c) \approx \pi_\theta(\cdot | x))$, and the distillation signal becomes weak. In practice, before running a long SDFT job, it is useful to inspect a small number of teacher-prompt completions and verify that greedy generations move toward the demonstrated tool-use pattern. If they do not, the likely issues are the demonstration quality, the prompt template, or the choice of base checkpoint rather than the distillation objective itself.

3.2 Forward vs. Reverse Kullback–Leibler divergence

Equation (4) writes the sequence-level objective with the student distribution in the first argument,

$$D_{\text{KL}}(\pi_\theta(\cdot | x) \| \pi_\phi(\cdot | x, c)).$$

However, the released `DistilTrainer` uses PyTorch’s `F.kl_div(input, target, log_target=True)` at the token level. Under the argument ordering used in the trainer, `input` corresponds to the student log-probabilities and `target` corresponds to the teacher log-probabilities. The resulting loss is therefore the teacher-weighted KL

$$D_{\text{KL}}(\pi_\phi(\cdot | h_t, c) \| \pi_\theta(\cdot | h_t))$$

at each unmasked completion position t , where $h_t = (x, y_{<t})$ denotes the student rollout prefix. Thus, with the default setting `alpha=0`, the implementation optimizes per-token forward KL from the teacher to the student. Setting `alpha=1` instead recovers the opposite direction, matching the ordering suggested by Equation (4). The repository README and the discussion in GitHub issue (hank0316 and idanshen, 2026) indicate that the reported runs used the forward-KL setting. Our reproduction therefore uses `alpha=0`, i.e. forward KL.

Interpreting inherited GRPO logs. The released trainer inherits several logging fields from the `trl` GRPO training loop, although SDFT does not optimize an RL reward. In our run, `main.py` does not pass reward functions to the trainer, so rewards and advantages are initialized to zero during generation. Consequently, `train/rewards = 0` in Weights & Biases is expected and should not be interpreted as a failed run (bhavitvyamalik, 2026). The relevant diagnostics are instead `train/loss`, `train/kl_approx`, and completion statistics such as `completions/mean_length`.

Reproduction Recipe. Table 2 gives the complete configuration used for the released checkpoint. We trained Qwen2.5-7B-Instruct with the released SDFT training entry point, `main.py`, on `data/tooluse_data/`. Unless otherwise stated, we kept the trainer defaults used by the public recipe and changed only the parameters listed in the table. The run used the forward-KL setting,

Table 2: Training and evaluation settings for our Tool Use replication.

Hyperparameter	Value
Base model	Qwen/Qwen2.5-7B-Instruct
Data	data/tooluse_data/ ($N_{\text{train}}=4046$, $N_{\text{eval}}=97$)
Learning rate	2×10^{-5}
LR schedule	cosine, warmup ratio 0.1
Training length	400 optimizer steps (≈ 3.2 epochs)
Optimizer	AdamW ($\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=10^{-8}$)
Per-device batch size	1
Gradient accumulation	32 prompts / step
Max prompt / completion length (train)	1024 / 1024
Max generation length (eval)	2048 (eval_tooluse.py)
Eval decoding	temperature 0 (greedy)
Teacher EMA	sync_ref_model=True, $\alpha_{\text{ref}}=0.01$, every step
KL loss	forward (alpha=0)
num_loss_tokens_to_skip	3
Seed	42

Table 3: Tool Use accuracy (official eval split). Eval: eval_tooluse.py, $T=0$, 2048 tokens.

Model	Accuracy (%)
Base Qwen2.5-7B-Instruct	42.2
SDFT, (Shenfeld et al., 2026) (reported)	70.6
SDFT, ours (step 400, seed 42)	70.1

alpha=0, an EMA teacher updated every step with $\alpha_{\text{ref}} = 0.01$, seed 42, and 400 optimizer steps. With an effective batch size of 32 prompts per optimizer step, this corresponds to approximately 3.2 passes over the 4,046-example Tool Use training split.

Training and evaluation use different length budgets. During training, we use a maximum prompt length of 1024 tokens and a maximum completion length of 1024 tokens. For evaluation, we use the released eval_tooluse.py script with greedy decoding, $T = 0$, and max_new_tokens=2048. We apply the same evaluation protocol to both the base model and the SDFT checkpoint. This detail is important: changing the decoding temperature or shortening the evaluation budget can change whether long tool-use traces are completed, and therefore such runs should not be mixed with the numbers reported in Table 3.

4 Results and training diagnostics

The main reproduction result is shown in Table 3. The base Qwen2.5-7B-Instruct model obtains 42.2% accuracy on the held-out Tool Use split. Using the reproduction recipe in Table 2, our SDFT checkpoint obtains 70.1%, an absolute improvement of 27.9 percentage points over the base model. This is within 0.5 percentage points of the 70.6% Tool Use result reported by Shenfeld et al. (Shenfeld et al., 2026).

We next inspect the training diagnostics from the same run. Unless otherwise stated, all curves correspond to the released checkpoint trained with forward KL, seed 42, and the configuration in Table 2.

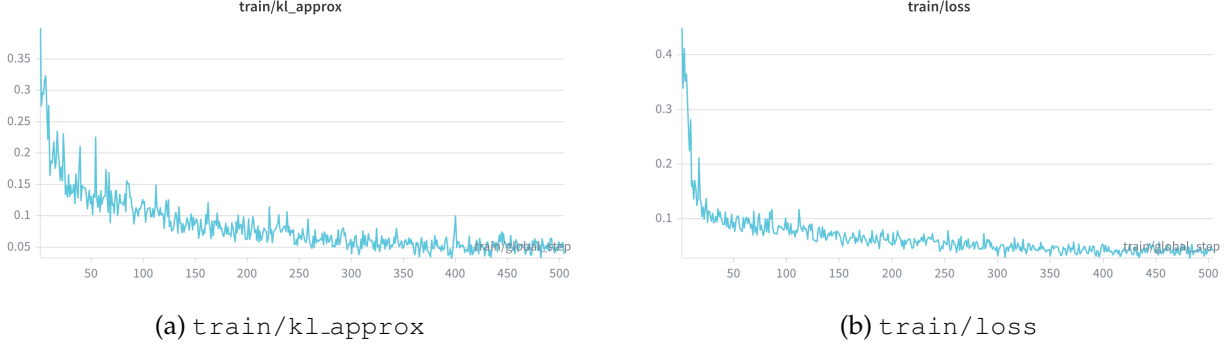


Figure 1: Teacher-student gap. **Left:** on-policy KL between student and EMA teacher. **Right:** distillation loss.

Teacher-student gap. The metric `train/kl_approx` is a Schulman-style on-policy approximation to $D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\phi})$, computed on sampled completion tokens (Schulman et al., 2017). In our run, this estimate decreases sharply during the first ~ 30 optimizer steps, indicating a rapid reduction in the teacher-student gap on the student-generated prefixes. It then continues to decrease more slowly before becoming visually flat after roughly step 300. At this point, the student and EMA teacher assign similar, but not identical, token distributions on the sampled rollouts. The distillation loss, `train/loss`, follows the same broad trend under the forward-KL objective, decreasing early and stabilizing later (Figure 1).

Why KL does not go to zero. We should not expect `train/kl_approx` to converge to zero, even in a healthy SDFT run. There are two reasons for this. First, the teacher is not fixed. At each optimizer step, its parameters are updated as an exponential moving average of the student,

$$\phi \leftarrow (1 - \alpha_{\text{ref}})\phi + \alpha_{\text{ref}}\theta, \quad \alpha_{\text{ref}} = 0.01.$$

Thus, π_{ϕ} is a lagged version of the evolving student rather than a stationary target. Second, the student and teacher are conditioned on different contexts: the student is rolled out from x , whereas the teacher is evaluated with the demonstration-conditioned prompt containing both x and c . A non-zero teacher-student gap can therefore remain even when the student has learned to produce rollouts that follow the demonstrated pattern. The plateau after roughly step 300 is best interpreted as a stable regime in which the student closely tracks, but does not exactly match, the EMA teacher under different conditioning contexts, rather than as evidence of failed optimization (Figure 1a).

Entropy. The entropy diagnostic provides a complementary view of the run. After the initial optimisation phase, `train/entropy` increases and then stabilises (Figure 2). We interpret this cautiously: the SDFT checkpoint does not appear to match the teacher by collapsing to a low-entropy token distribution. Rather, after the early reduction in teacher-student KL, the student maintains non-trivial token-level uncertainty on its sampled tool-use prefixes. This behaviour is consistent with a stable distillation regime rather than degenerate optimization.

5 Artifacts

To make the reproduction easy to inspect and build on, we release the trained checkpoint together with the evaluation output and training diagnostics:

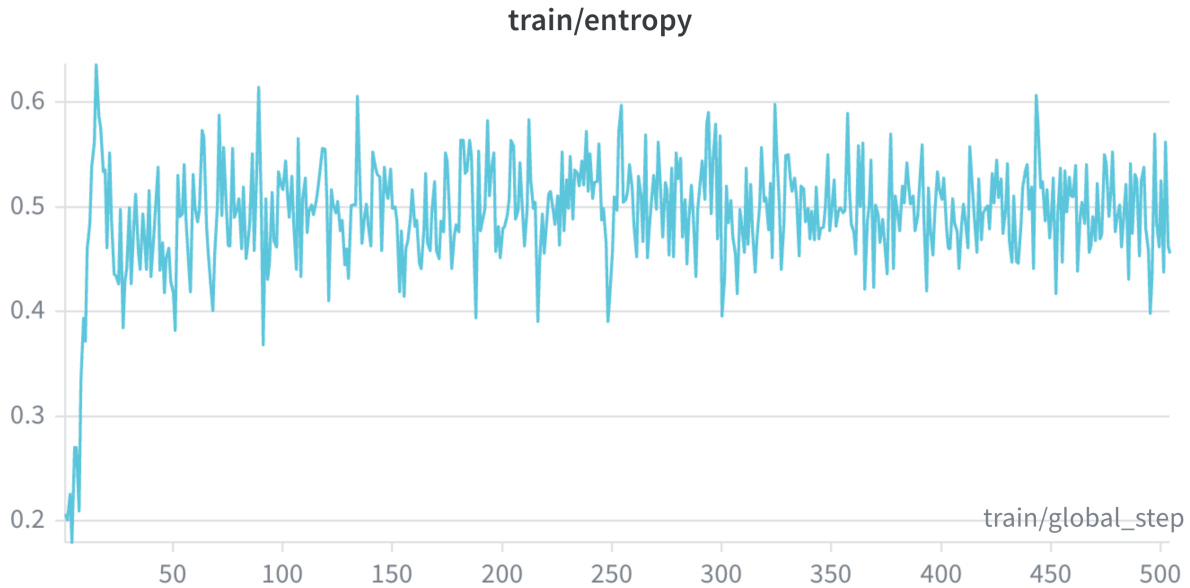


Figure 2: train/entropy vs. optimizer step.

- **Fine-tuned checkpoint:** the seed-42, 400-step SDFT model is available on Hugging Face at <https://huggingface.co/KickItLikeShika/qwen-2.5-7b-instruct-sdft-tooluse>.
- **Evaluation output:** the corresponding `eval_results.json` file is included in the same Hugging Face repository.
- **Training logs:** the Weights & Biases report is available at: <https://api.wandb.ai/links/egyttsteam/d97ty5d9>.
- **Diagnostic curves:** we also release the training curves used to produce Figures 1–2.

For consistency with the numbers reported in this note, checkpoints should be evaluated with the released `eval_tooluse.py` script using greedy decoding, $T = 0$, and `max_new_tokens=2048`. The implementation details in Section 3 and the hyperparameters in Table 2 define the full reproduction recipe for the released checkpoint.

6 Limitations, Future Updates, and Conclusions

This report is intentionally narrow. Our goal is to document a concrete reproduction of the Tool Use result and to release a working checkpoint, not to provide a comprehensive evaluation of SDFT across all settings. The main result uses one base model, Qwen2.5-7B-Instruct, one task split, one random seed, and the released Tool Use evaluation protocol. The held-out split is also small, with $N_{\text{eval}} = 97$, so small changes in the number of correct examples can noticeably affect the reported percentage.

We therefore do not claim that the same hyperparameters will reproduce all results from the original SDFT paper or transfer unchanged to other model families, datasets, or domains. In

particular, the effectiveness of SDFT depends on the quality of the demonstration c , the teacher-prompt template, the ability of the base model to use the demonstration in context, and the exact evaluation protocol. These details may interact differently across tasks.

This document should be read as a reproducibility note for successful Tool Use runs. We plan to update the report as we obtain additional results, including further ablations over the KL direction and teacher updates, and reproductions on additional tasks or base models. We will also update the released artefacts when new checkpoints, logs, or evaluation files become available.

Despite these limitations, the reproduction is encouraging. Using the recipe documented in this note, we reproduce the SDFT Tool Use result with Qwen2.5-7B-Instruct, obtaining 70.1% accuracy compared with the 70.6% reported result and 42.2% for the base model. The reproduction required matching several implementation details that are easy to miss from the high-level method description alone: the demonstration-conditioned teacher prompt, the per-token forward-KL setting used by the released trainer, the EMA teacher update, skipped completion tokens, and the greedy 2048-token evaluation protocol.

The purpose of this note is to make that recipe explicit and useful to others. We release the fine-tuned checkpoint, evaluation output, training logs, and diagnostic curves so that future work can inspect, reuse, and build on a working SDFT reproduction. More broadly, our results suggest that the Tool Use experiment is reproducible when the executable recipe is matched carefully, while also highlighting the importance of documenting implementation-level choices in self-distillation and on-policy training methods.

References

- Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes, 2024a. URL <https://arxiv.org/abs/2306.13649>.
- Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes, 2024b. URL <https://arxiv.org/abs/2306.13649>.
- bhavitvyamalik. Reward stays at 0 and eventually loss becomes 0 too. GitHub issue, <https://github.com/idanshen/Self-Distillation/issues/14>, 2026.
- devonbrackbill. Unable to reproduce reported results for Qwen2.5-7B-Instruct. GitHub issue, <https://github.com/idanshen/Self-Distillation/issues/9>, 2026. Open reproducibility thread on the official SDFT repository.
- hank0316 and idanshen. Default to forward KL? GitHub issue, <https://github.com/idanshen/Self-Distillation/issues/5>, 2026. Author confirms paper experiments used per-token forward KL ($\alpha=0$).
- Yoon Kim and Alexander M. Rush. Sequence-level knowledge distillation, 2016. URL <https://arxiv.org/abs/1606.07947>.
- Proteusiq. Reproducibility caveats for sdft: Directions vs. magnitudes. GitHub issue, <https://github.com/Proteusiq/unthinking/issues/86>, 2026. Summarizes community replication gaps including issues #5, #9, and #15.

John Schulman, Sergey Levine, Philipp Moritz, Michael I. Jordan, and Pieter Abbeel. Trust region policy optimization, 2017. URL <https://arxiv.org/abs/1502.05477>.

Idan Shenfeld, Mehul Damani, Jonas Hübötter, and Pulkit Agrawal. Self-distillation enables continual learning, 2026. URL <https://arxiv.org/abs/2601.19897>.

yufeng-04. Unable to reproduce reported results for Olmo3-7B-think on medical task. GitHub issue, <https://github.com/idanshen/Self-Distillation/issues/15>, 2026.

Matthieu Zimmer, Xiaotong Ji, Tu Nguyen, and Haitham Bou Ammar. Rethinking large language model distillation: A constrained markov decision process perspective, 2025. URL <https://arxiv.org/abs/2509.22921>.